VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belgavi-590014

Computer Graphics and Fundamentals of Image Processing Mini Project Report on

"RUBIK'S CUBE"

Submitted to

Visvesvaraya Technological University

In the partial fulfillment of requirements for the award of degree

Bachelor of

Engineering

In

COMPUTER SCIENCE AND ENGINEERING

Submitted By

SURAJ B **4RA22CS413**

KIRAN KUMAR C V **4RA22CS405**

Under The Guidance of

Mr. SANJAY M

Assistant Professor, Dept. of CSE

Rajeev Institute of Technology,

Hassan

RAJEEV INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

HASSAN-573201

2023-2024

RAJEEV INSTITUTE OF TECHNOLOGY, HASSAN

(Approved by AICTE, New Delhi and Affiliated to VTU, Belagavi.)

Plot #1-D, Growth Centre, Industrial Area, B-M Bypass Road, Hassan – 573201 Ph:(08172)-243180/83/84 Fax:(08172)-243183

Department of Computer Science and Engineering

CERTIFICATE

Certified that the **Computer Graphics and Fundamentals of Image Processing Mini Project** entitled **"RUBIK'S CUBE"** is carried out by Mr.SURAJ B [4RA22CS413], Mr.KIRAN KUMAR C V [4RA22CS405] respectively, a bonafide students of **RAJEEV INSTITUTE OF TECHNOLOGY**, Hassan in partial fulfillment for the award of **BACHELOR OF ENGINEERING** in **COMPUTER SCIENCE AND ENGINEERING** of the Visvesvaraya Technological University, Belagavi during the year 2023-2024. The Project Phase-1 report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said Degree.

Mr. SANJAY M	**Dr. SHRISHAIL MATH**	**Dr. MAHESH P K**
Assistant Professor	Head of the Department	Principal
Dept. of Computer Science & Engineering	Computer Science & Engineering	**RIT, HASSAN**
RIT, HASSAN	**RIT, HASSAN**	

Name of the examiners — Signature with date

1. ....
2. ....

DECLARATION

We **SURAJ B, KIRAN KUMAR C V** bearing USN **4RA22CS413**, **4RA22CS405,** students of 6th semester B.E in **Computer Science and Engineering**, **Rajeev Institute of Technology, Hassan**, hereby declare that the work being presented in the dissertation entitled "**RUBIK'S CUBE**" has been carried out by us under the supervision of guide **Mr. SANJAY M**, Assistant Professor, Computer Science and Engineering, **Rajeev Institute of Technology, Hassan**, as partial fulfillment of requirement for the award of B.E Degree of **Bachelor of Engineering in Computer Science and Engineering at Visvesvaraya Technological University, Belagavi** is and authentic record of my own carried out by us during the academic year 2023-2024.

SURAJ B
4RA22CS413

KIRAN KUMAR C V
4RA22CS405

PLACE:
HASSAN
DATE:
.....

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

We would like to profoundly thank our **Management of Rajeev Institute of Technology** & our President **Dr. Rachana Rajeev** for providing such a healthy environment.

We would like to express our sincere thanks to our principal **Dr. Mahesh P K**, Rajeev Institute of Technology for his encouragement that motivated us for successful completion of project work.

We wish to express our gratitude to **Dr. Shrishail Math**, Head of the Department of Information Science & Engineering for providing a good working environment and for his constant support and encouragement.

It gives us great pleasure to express our gratitude to **Mr. Sanjay M** Assistant Professor, Department of Computer Science & Engineering for her expert guidance, initiative and encouragement that led us to complete this project.

We would also like to thank all our staffs of Information Science and Engineering department who have directly or indirectly helped us in the successful completion of this phase-1 project and also, we would like to thank our parents.

SURAJ B
4RA22CS413

KIRAN KUMAR C V
4RA22CS405

ABSTRACT

This project presents the development of an interactive Rubik's Cube simulation using OpenGL, aiming to create a visually accurate and fully functional digital representation of the classic 3x3 Rubik's Cube puzzle. By leveraging the power of OpenGL for rendering, the simulation is designed to provide smooth and realistic graphics, allowing users to rotate and manipulate the cube seamlessly in a 3D space. The project features a user-friendly interface that facilitates intuitive control over the cube's movements, employing keyboard and mouse inputs to rotate individual faces of the cube, thereby mimicking the physical interactions of the real-world puzzle. In addition to basic manipulation, the simulation incorporates scrambling algorithms, solution visualization, and performance optimizations to ensure an engaging and responsive user experience. Throughout the development process, various aspects of computer graphics are explored, including transformations, lighting, and shading, as well as the mathematical principles underlying the Rubik's Cube mechanics. The final product is an educational and entertaining tool that not only showcases the capabilities of OpenGL but also provides a platform for further exploration into graphics programming and algorithm design. This project serves as a testament to the intricate blend of technical prowess and creative problem-solving required to bring a complex and beloved puzzle into the digital realm.

CONTENTS

DECLARATION	i
ACKNOLEDGEMENT	ii
ABSTRACT	iii
CONTENTS	iv
Chapter Name	**Page no**
1.INTRODUCTION	**1-3**
1.1 Computer Graphics	1
1.2 OpenGL Interface	1-2
1.3 OpenGL Overview	2-3
2.SYSTEM REQUIREMENTS	**4**
2.1 Software Requirements	4
2.2 Hardware Requirements	4
2.3 Functional Requirements	4-5
3. ABOUT THE PROJECT	**6-11**
3.1 Overview	6
3.2 User Interface	6-7
3.3 Design	7
3.4 Algorithm	7-10
3.5 Flow of the Program	10
3.6 Purpose	11
3.7 Scope	11
4. TESTING	**12**
5. SNAPSHOTS	**13-14**
6. CONCLUSION	**15**
REFERNCES	**16**

CONTENTS

LIST OF FIGURES

Figure No.	Name of the Figure	Page No.
1.1	Basic OpenGL Operation	3
3.3	Data flow diagram	7
4.1	Rubik's cube	13
4.2	Rubik's cube Top	13
4.3	Right Inverted	14
4.4	Rubik's cube Front	14

CHAPTER 6

CONCLUSION

Creating a Rubik's Cube using OpenGL involves several key steps. First, you need to understand OpenGL and model the cube as 27 smaller cubes in a 3x3x3 grid. Each face must be correctly colored or textured. Handling user input allows for rotating the cube and its faces, translating mouse movements into rotations. Managing the cube's logical state ensures rotations are accurate and solvable. Smooth animations improve user experience, and optimization keeps the application running well. Testing and debugging are crucial for handling edge cases and ensuring everything works correctly. This project teaches valuable lessons in 3D graphics, mathematical transformations, and interactive application development.

REFERENCES

[1] OpenGL Programming Guide (Addison-Wesley Publishing Company)

[2] The OpenGL Utility Toolkit (GLUT)Programming Interface

-API Version 3 BY MARK J. KILGARD

[3] Interactive computer graphics

-A top down approach by using Open GL by EDWARD ANGEL

CHAPTER 1

INTRODUCTION

1.1 Computer Graphics:

Graphics provides one of the most natural means of communicating within a computer, since our highly developed 2D and 3D pattern-recognition abilities allow us to perceive and process pictorial data rapidly and effectively. Interactive computer graphics is the most important means of producing pictures since the invention of photography and television. It has the added advantage that, with the computer, we can make pictures not only of concrete real world objects but also of abstract, synthetic objects, such as mathematical surfaces and of data that have no inherent geometry, such as survey results.

Computer graphics started with the display of data on hardcopy plotters and cathode ray tube screens soon after the introduction of computers themselves. It has grown to include the creation, storage, and manipulation of models and images of objects. These models come from a diverse and expanding set of fields, and include physical, mathematical, engineering, architectural, and even conceptual structures, natural phenomena, and so on. Computer graphics today is largely interactive. The user controls the contents, structure, and appearance of the objects and of their displayed images by using input devices, such as keyboard, mouse, or touch-screen.

Graphics today is used in many different areas. Graphics provides one of the most natural means of communicating within a computer, since our highly developed 2D and 3D pattern-recognition abilities allow us to perceive and process pictorial data rapidly and effectively. Interactive computer graphics is the most important means of producing pictures since the invention of photography and television. It has the added advantage that, with the computer, we can make pictures not only of concrete real world objects but also of abstract, synthetic objects, such as mathematical surfaces and of data that have no inherent geometry, such as survey results.

OpenGL is an application program interface (API) offering various functions to implement primitives, models and images. This offers functions to create and manipulate render lighting, coloring, viewing the models. OpenGL offers different coordinate system and frames. OpenGL offers translation, rotation and scaling of objects. Due to close relationships between the input devices and the display, the handling of such devices is included in the study of computer graphics. The advantages of the interactive graphics are many in number. Graphics provides one of the most natural means of communicating with a computer.

1.2 OpenGL Interface:

OpenGL is an application program interface (API) offering various functions to implement primitives, models and images. This offers functions to create and manipulate render lighting, coloring, viewing the models. OpenGL offers different coordinate system and frames. OpenGL offers translation, rotation and scaling of objects.

Most of our applications will be designed to access OpenGL directly through functions in three libraries. They are:

1. **Main GL:** Library has names that begin with the letter gl and are stored in a library usually referred to as GL.

2. **OpenGL Utility Library (GLU):** This library uses only GL functions but contains code for creating common objects and simplifying viewing.

3. **OpenGL Utility Toolkit (GLUT):** This provides the minimum functionality that should be accepted in any modern windowing system.

1.3OpenGL Overview:

OpenGL(Open Graphics Library) is the interface between a graphic program and graphics hardware. ***It is streamlined.*** In other words, it provides low-level functionality. For example, all objects are built from points, lines and convex polygons. Higher level objects like cubes are implemented as six four-sided polygons.

OpenGL supports features like 3-dimensions, lighting, anti-aliasing, shadows, textures, depth effects, etc

It is system-independent. It does not assume anything about hardware or operating system and is only concerned with efficiently rendering mathematically described scenes. As a result, it does not provide any windowing capabilities.

It is a state machine. At any moment during the execution of a program there is a current model transformation

It is a rendering pipeline. The rendering pipeline consists of the following steps:

- Defines objects mathematically.
- Arranges objects in space relative to a viewpoint.
- Calculates the color of the objects.
- Rasterizes the objects. OpenGL Pipeline

The OpenGL architecture is structured as a state-based pipeline. Below is a simplified diagram of this pipeline. Commands enter the pipeline from the left.

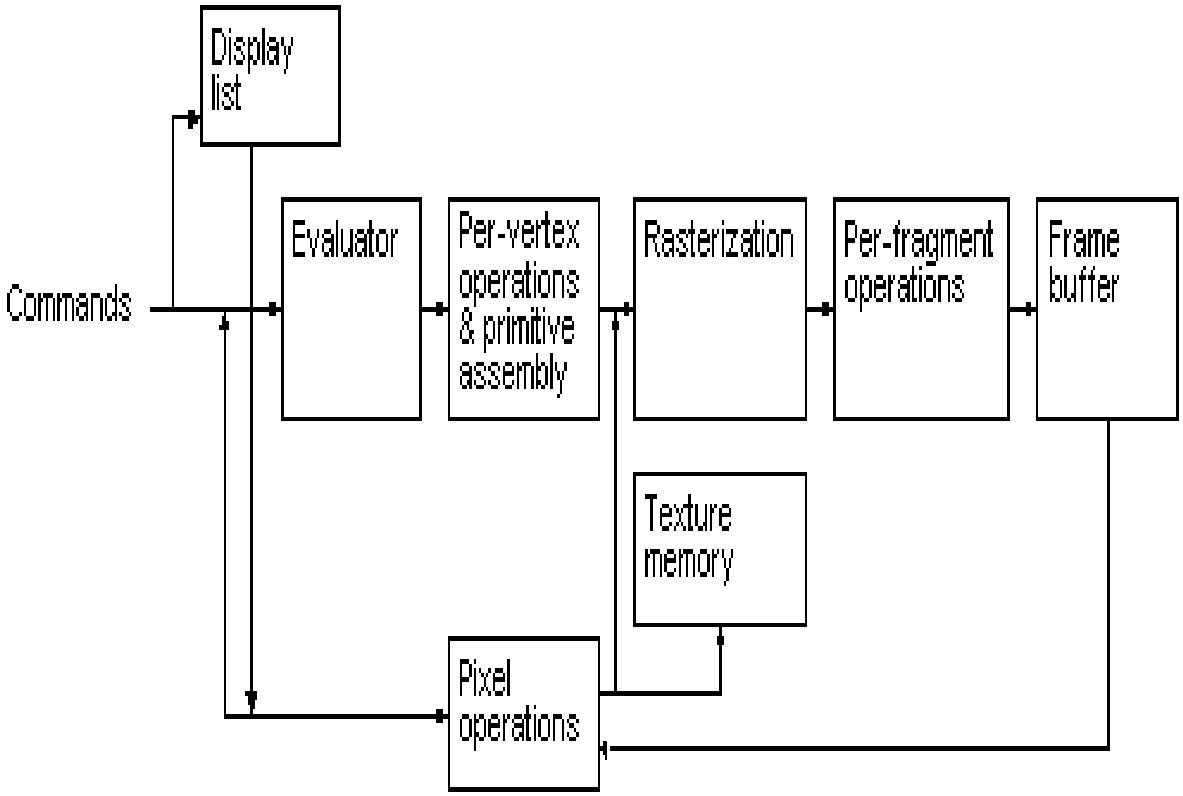


Fig 1.1: Basic OpenGL Operation

Commands may either be accumulated in display lists, or processed immediately through the pipeline. Display lists allow for greater optimization and command reuse, but not all commands can be put in display lists.

The first stage in the pipeline is the evaluator. This stage effectively takes any polynomial evaluator commands and evaluates them into their corresponding vertex and attribute commands.

The second stage is the per-vertex operations, including transformations, lighting, primitive assembly, clipping, projection, and viewport mapping.

The third stage is rasterization. This stage produces fragments, which are series of framebuffer addresses and values, from the viewport-mapped primitives as well as bitmaps and pixel rectangles.The fourth stage is the per-fragment operations. Before fragments go to the

CHAPTER 2

SYSTEM REQUIREMENTS

2.1 HARDWARE REQUIREMENTS:

- Dual Core Processor
- 2GB RAM
- 40GB Hard disk
- DVD drive
- Mouse and other pointing devices
- Keyboard

2.2 SOFTWARE REQUIREMENTS:

- Programming language – C/C++ using OpenGL
- Operating system – Linux operating system
- Compiler – C Compiler
- Graphics library – GL/glut.h
- OpenGL 2.0

2.3 FUNCTIONAL REQUIREMENTS:

OpenGL APIs:

If we want to have a control on the flow of program and if we want to interact with the window system then we use OpenGL API'S. Vertices are represented in the same manner internally, whether they are specified as two-dimensional or three-dimensional entities, everything that we do are here will be equally valid in three dimensions. Although OpenGL is easy to learn, compared with other APIs, it is nevertheless powerful. It supports the simple three dimensional programs and also supports the advanced rendering techniques.

GL/glut.h:

We use a readily available library called the OpenGL Utility Toolkit (GLUT), which provides the minimum functionality that should be expected in any modern windowing system. The application program uses only GLUT functions and can be recompiled with the GLUT library for other window system. OpenGL makes a heavy use of macros to increase code readability and avoid the use of magic numbers. In most implementation, one of the include lines

CHAPTER 3

ABOUT THE PROJECT

3.1 OVERVIEW:

Creating a Rubik's Cube in OpenGL involves combining knowledge of the Rubik's Cube mechanics and OpenGL programming. A Rubik's Cube is a 3x3x3 puzzle with six colored faces, requiring rotations to achieve a uniform color on each face. OpenGL, a cross-platform API for rendering 2D and 3D graphics, can be used to simulate this puzzle. Setting up an OpenGL environment typically involves libraries like GLFW for window management and GLEW for handling extensions. The process includes initializing OpenGL, creating and rendering the 27 smaller cubes (cubies) that form the Rubik's Cube, and implementing rotation algorithms to manipulate the cube's faces, allowing for user interaction and solving visualization..

3.2 USER INTERFACE:

- **file_gen:** is used to check whether high score file already exists. If it does not, it will create. If it does, it will read data from existing file.
- **render_header:** to display the player's name along with a running timer. The timer gets its values from the clock class, which maintains the time elapsed since the game started in hour, minutes and seconds.
- **hiscore_check:** to determine the position of the player in the highscores list. If he completes the game in a time lesser than the top 10 players, it implements an insertion sort to insert the player's name and time into the existing high score list.
- **write_file:** is used to write a text file of the current player class. The player class contains details of the player name and the time he took to complete the game. All these details, along with the pre-existing records of the highscores are arranged in ascending order of time and are written into a text file.
- **Sub:** is the display function for the sub window created to display the highscores. This window is activated when the user chooses the option "highscores" from the menu obtained on right click of the mouse.

Mouse events: When mouse event occurs, the ASCII code for the corresponding coordinates that generate event and the location of mouse are returned.Mouse callback function is

```
glutMouseFunc (mouse);
Void mouse (int btn, int state, int x, int y)
{
if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
begin = x;

}
```

Menu Entry: GLUT provides one additional feature, pop_up menus, which we can use with the mouse to create sophisticated interactive application

```
glutCreateMenu ();

glutAddMenuEntry ();

glutAttachMenu (GLUT_RIGHT_BUTTON);
```

3.3 DESIGN:

The flow chart describes how the path travels.

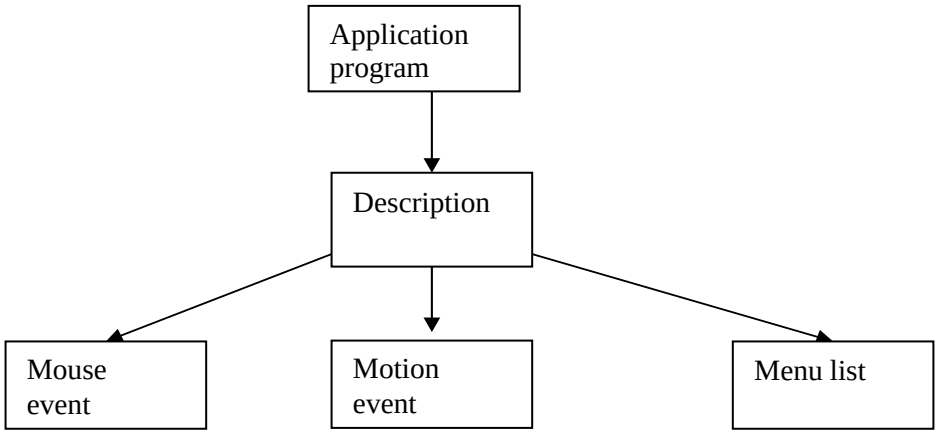


Fig 3.3: Data flow diagram.

❖ **Figure 3.3:** under Implementation Chapter explains the Data Flow Diagram.This includes the flow from application program to handling different events like Mouse event, Motion event and the Menu list.

3.3 ALGORITHM:

STEP 1: Define vertices for 27 cubes which are used to compose one whole cube known as "Rubiks cube".

STEP 2: Define colors for each cube of the Rubiks cube to distinguish one cube from the other and as well as color for the speed meter, which is used to control the speed of rotation.

Output( ) function:

STEP 3: This function is used to display the message using the Commands glutBitmapCharacter( ) and glRasterpos*( ).

Polygon( ) function:

STEP 4: Draw a polygon via list of vertices with the line of 3 pixels wide.

Colorcube( ) function:

STEP 5: Map vertices to faces. Hence the use of color cube function has done 27 times with different prefixes for all the 27 cubes amongst which one with no color, 6 with one color, 12 with two colors and 8 with three colors on it.

Speedmeter( ) function:

STEP 6: This function is used to define vertices for a speed meter, which is used to control the speed of rotation.

Display( ) function:

STEP 7: Clear the frame buffers and the z-buffer.

STEP 8: Invoke the function speed meter and output.

STEP 9: Using the variables rotation and in versethe rotation for the faces are defined, where if rotation is one and the inverse flag is zero then the top face of the cube will be rotated in the clockwise direction and incase if rotation is one and also is the inverse flag then the top face of the cube will be rotated in the anti-clockwise direction.

STEP 10: The same procedure that has been specified in the step 9 is adopted with different values for the rotation, such as rotation with the value two is implemented for right three for front, four for left, five for back and six for bottom rotations respectively with the implementation of inverse variable being same.

STEP 11: Invoke the output function and swap the buffers.

Transpose( ) function:

STEP 12: This function is used to define the transpose for all the six faces.

Topc( ) function:

STEP 13: This function is used to assign the values when the operation is rotation of the top.

Frontc( ) function:

STEP 14: This function is used to assign the values when the operation is rotation of the front face.

Rightc( ) function:

STEP 15: This function is used to assign the values when the operation is rotation of the right face.

Leftc( ) function:

STEP 16: This function is used to assign the values when the operation is rotation of the left face.

Backc( ) function:

STEP 17: This function is used to assign the values when the operation is rotation of the back face.

Bottomc( ) function:

STEP 18: This function is used to assign the values when the operation is rotation of the bottom face.

Spincube( ) function:

STEP 19: This is function is an idle callback which rotates the cube accordingly, if rotation is one and inverse is zero then rotate the top face of the cube by 90 degrees in the clockwise direction and if inverse is one then rotate the same face by 90 degree in the anti-clockwise direction.

STEP 20: The same procedure is implemented for the other faces of the cube with different rotation values.

STEP 21: The cube is redisplayed after the rotation.

Motion( ) function:

STEP 22: This is used to rotate the cube about the selected axis.

Mouse( ) function:

STEP 23: Mouse callback function. This allows user to give input through mouse buttons.

Keyboard( ) function:

STEP 24: use the keys 'a','s','d','f','g','h' to rotate the faces of the accordingly in the clockwise direction and the keys 'q','w','e','r','t','y' to rotate in the anti-clockwise direction.

STEP 25: use the keys '1','2','4','5','6','8','9' are used to move the viewer along the axis.

STEP 26: use the keys 'm' and 'n' to alter the value of the rotation meter which controls the speed of the rotation.

STEP 27: use the key 'o' for the automatic solving of the cube.

Myreshape( ) function:

STEP 28: Define a viewport and set the matrix to projection and modelview matrices.

Mymenu( ) function:

STEP 29: Define the actions corresponding to each entry in the menu.

Main( ) function:

STEP 30: Both double and z-buffer is enabled. Invoke the start function.

STEP 31: Add entries to the menu and link the menu to the right mouse button.

STEP 32: Stop.

3.5 FLOW OF THE PROGRAM:

1. The interaction between the windowing system and OPENGL is initiated.We initialize the window size and window position.
2. Display function is called where the functions and operations for the Rubiks cube are defined.
3. Using the left mouse button the cube can be rotated along the required axis.
4. Using the right mouse button menus can be viewed, by selecting the options from the menu the required face of the cube can be rotated either clockwise or anti-clockwise.
5. The rotation of faces of the cube can also be done using keys from the keyboard.
6. Press the key 'a' to rotate the top face of the cube in the clockwise direction.
7. Press the key 'q' to rotate the top face of the cube in the anti-clockwise direction.
8. Press the key 's' to rotate the right face of the cube in the clockwise direction.
9. Press the key 'w' to rotate the right face of the cube in the anti-clockwise direction.
10. Press the key 'd' to rotate the front face of the cube in the clockwise direction.
11. Press the key 'e' to rotate the front face of the cube in the anti-clockwise direction.
12. Press the key 'f' to rotate the left face of the cube in the clockwise direction.
13. Press the key 'r' to rotate the left face of the cube in the anti-clockwise direction.
14. Press the key 'g' to rotate the back face of the cube in the clockwise direction.
15. Press the key 't' to rotate the back face of the cube in the anti-clockwise direction.

3.6 PURPOSE:

Creating a Rubik's Cube in OpenGL serves as a comprehensive learning experience, providing hands-on practice with key concepts like transformations, lighting, and shading. It aids in modeling complex 3D objects, learning animation techniques, and implementing user interactions through event handling. Additionally, it presents a challenging problem that combines graphics programming and algorithm design, especially if solving the cube is involved. Overall, it is an excellent project for deepening understanding of computer graphics and interactive applications.

3.7 SCOPE:

The scope of a Rubik's Cube in the context of OpenGL and graphics programming includes creating a visually accurate 3D model, implementing rotations and movements using matrix operations, and developing smooth animations for interactions. It also involves enabling user inputs to manipulate the cube, applying rendering techniques like lighting, shading, and texture mapping for visual realism, and designing algorithms to solve the cube. This project encompasses a range of skills from basic 3D modeling and transformations to advanced user interaction and problem-solving algorithms.

CHAPTER 4

TESTING

Testing process started with the testing of individual program units such as functions or objects. These were then integrated into sub-systems and systems, and interactions of these units were tested.

Testing involves verification and validation.

Validation: "Are we building right product?"

Verification: "Are we building the product right?"

The ultimate goal of the verification and validation process is to establish confidence that the software system is 'fit for purpose'. The level of required confidence depends on the system's purpose, the expectations of the system users and the current marketing environment for the system.

With the verification and validation process, there are two complementary approaches to the system checking and analysis:

Software inspections or peer reviews analyses and check system representations such as the requirements document, design diagrams, and the program source code. Software testing involves running an implementation of the software with test data.

CHAPTER 5

SNAPSHOTS

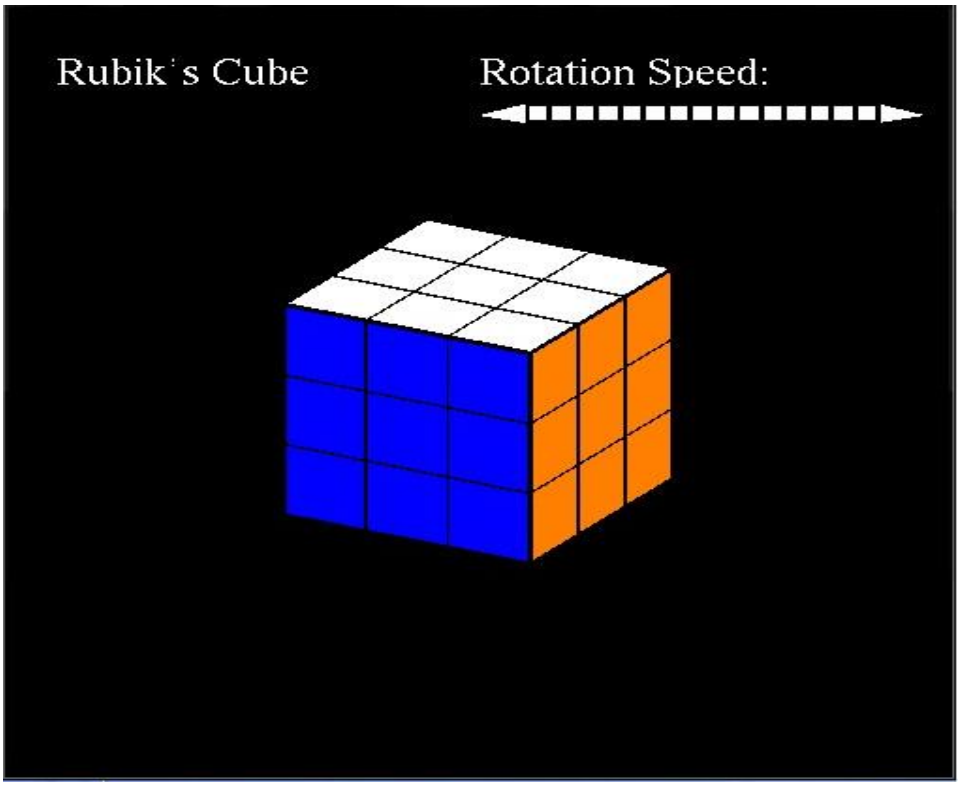


Fig 4.1: Rubik's cube

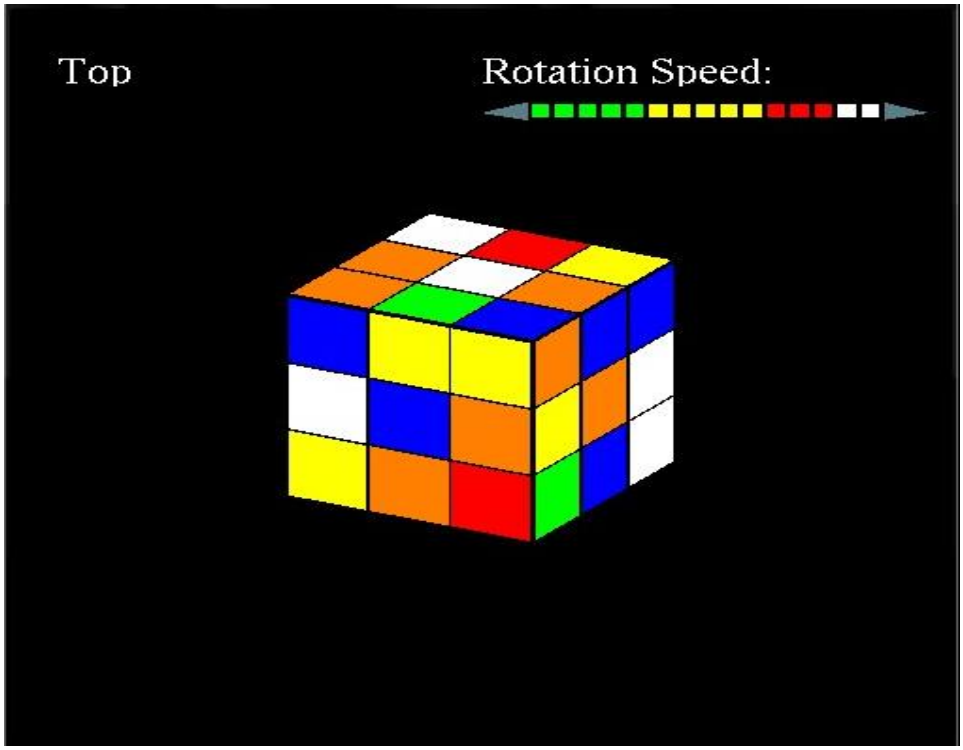


Fig 4.2: Rubik's cube Top

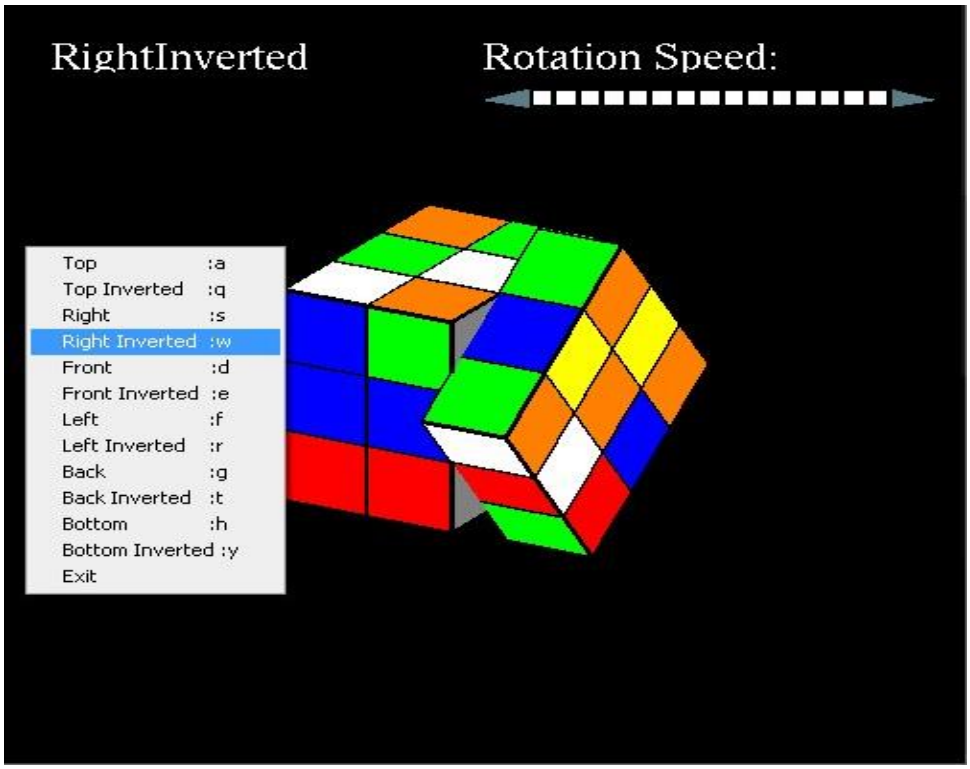


Fig 4.3: Right Inverted

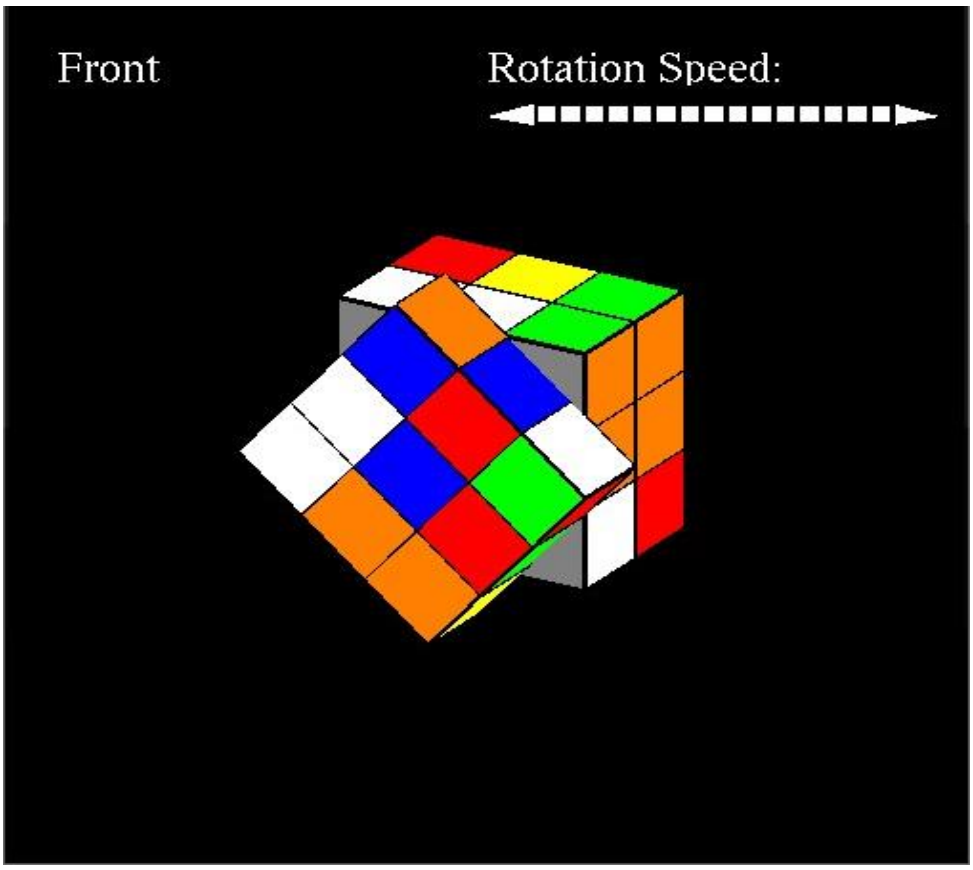


Fig 4.4: Rubik's cube Front